```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <limits>
using namespace std;

// Base class
class Person {
protected:
    int id;
    string name;

public:
    Person(int i = 0, string n = "")
        : id(i), name(n) {}

    virtual void display() const {
        cout << "ID: " << id << "\nName: " << name << '\n';
    }

    virtual ~Person() = default;

    int getId() const { return id; }
    string getName() const { return name; }
};

// Derived class
class Member : public Person {
private:
```

```
    string branch, club, role;

public:
    Member(int i = 0, string n = "", string b = "",
           string c = "", string r = "Member")
        : Person(i, n), branch(b), club(c), role(r) {}

    void display() const override {
        cout << "Member ID: " << id
             << "\nName: " << name
             << "\nBranch: " << branch
             << "\nClub: " << club
             << "\nRole: " << role << "\n";
    }

    string getBranch() const { return branch; }
    string getClub() const { return club; }
    string getRole() const { return role; }
};

class Club {
private:
    int id;
    string name, coordinator, category;

public:
    Club(int i = 0, string n = "", string co = "",
         string ca = "")
        : id(i), name(n), coordinator(co), category(ca) {}

    int getId() const { return id; }
```

```
    string getName() const { return name; }

    void display() const {
        cout << "Club ID: " << id
             << "\nClub Name: " << name
             << "\nCoordinator: " << coordinator
             << "\nCategory: " << category << "\n";
    }
};

class Event {
private:
    int id;
    string name, club, date, venue;

public:
    Event(int i = 0, string n = "", string c = "",
          string d = "", string v = "")
        : id(i), name(n), club(c), date(d), venue(v) {}

    int getId() const { return id; }
    string getName() const { return name; }
    string getClub() const { return club; }

    void display() const {
        cout << "Event ID: " << id
             << "\nEvent Name: " << name
             << "\nClub: " << club
             << "\nDate: " << date
             << "\nVenue: " << venue << "\n";
    }
```

```
};

struct Registration {
    int memberId, eventId;
    bool present;

    Registration(int m = 0, int e = 0, bool p = false)
        : memberId(m), eventId(e), present(p) {}
};

class CollegeClubSystem {
private:
    vector<Club> clubs;
    vector<Member> members;
    vector<Event> events;
    vector<Registration> registrations;

    // Input helpers
    int readInt(const string& message) {
        int value;
        while (true) {
            cout << message;
            if (cin >> value) {
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                return value;
            }
            cout << "Please enter a valid number.\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }
    }
```

```
string readText(const string& message) {
    string value;
    cout << message;
    getline(cin, value);
    return value;
}

bool clubExists(const string& name) const {
    for (const auto& c : clubs)
        if (c.getName() == name) return true;
    return false;
}

int memberIndex(int id) const {
    for (int i = 0; i < (int)members.size(); i++)
        if (members[i].getId() == id) return i;
    return -1;
}

int eventIndex(int id) const {
    for (int i = 0; i < (int)events.size(); i++)
        if (events[i].getId() == id) return i;
    return -1;
}

bool registered(int mid, int eid) const {
    for (const auto& r : registrations)
        if (r.memberId == mid && r.eventId == eid)
            return true;
    return false;
```

```
}

// File handling
void saveData() const {
    ofstream f1("clubs.txt");
    for (const auto& c : clubs)
        f1 << c.getId() << '|' << c.getName() << '\n';

    ofstream f2("members.txt");
    for (const auto& m : members)
        f2 << m.getId() << '|' << m.getName() << '|'
           << m.getBranch() << '|' << m.getClub() << '|'
           << m.getRole() << '\n';

    ofstream f3("events.txt");
    for (const auto& e : events)
        f3 << e.getId() << '|' << e.getName() << '|'
           << e.getClub() << '\n';

    ofstream f4("registrations.txt");
    for (const auto& r : registrations)
        f4 << r.memberId << '|' << r.eventId << '|'
           << r.present << '\n';
}

void loadData() {
    string line, a, b, c, d, e;
    int id, mid, eid, present;

    ifstream f1("clubs.txt");
    while (getline(f1, line)) {
```

```
    stringstream ss(line);
    if (getline(ss, a, '|') && getline(ss, b, '|')) {
        try { clubs.emplace_back(stoi(a), b, "", ""); }
        catch (...) {}
    }
}

ifstream f2("members.txt");
while (getline(f2, line)) {
    stringstream ss(line);
    if (getline(ss, a, '|') && getline(ss, b, '|')
        && getline(ss, c, '|') && getline(ss, d, '|')
        && getline(ss, e)) {
        try {
            members.emplace_back(stoi(a), b, c, d, e);
        } catch (...) {}
    }
}

ifstream f3("events.txt");
while (getline(f3, line)) {
    stringstream ss(line);
    if (getline(ss, a, '|') && getline(ss, b, '|')
        && getline(ss, c, '|')) {
        try { events.emplace_back(stoi(a), b, c, "", ""); }
        catch (...) {}
    }
}

ifstream f4("registrations.txt");
while (getline(f4, line)) {
```

```
        stringstream ss(line);
        if (getline(ss, a, '|') && getline(ss, b, '|')
            && getline(ss, c)) {
            try {
                registrations.emplace_back(
                    stoi(a), stoi(b), stoi(c) != 0);
            } catch (...) {}
        }
    }
}

void addClub() {
    int id = readInt("Enter Club ID: ");
    if (any_of(clubs.begin(), clubs.end(),
        [id](const Club& c) { return c.getId() == id; })) {
        cout << "Club ID already exists.\n";
        return;
    }

    string name = readText("Enter Club Name: ");
    string coordinator = readText("Faculty Coordinator: ");
    string category = readText("Category: ");

    clubs.emplace_back(id, name, coordinator, category);
    saveData();
    cout << "Club added successfully!\n";
}

void showClubs() const {
    if (clubs.empty()) {
        cout << "No clubs available.\n";
```

```
        return;
    }
    for (const auto& c : clubs) {
        c.display();
        cout << "--------------------\n";
    }
}

void addMember() {
    int id = readInt("Enter Member ID: ");
    if (memberIndex(id) != -1) {
        cout << "Member ID already exists.\n";
        return;
    }

    string name = readText("Enter Student Name: ");
    string branch = readText("Enter Branch: ");
    string club = readText("Enter Club Name: ");

    if (!clubExists(club)) {
        cout << "Club not found. Please add the club first.\n";
        return;
    }

    string role = readText("Enter Role (Member/Coordinator/President): ");

    members.emplace_back(id, name, branch, club, role);
    saveData();
    cout << "Member added successfully!\n";
}
```

```
void showMembers() const {
    if (members.empty()) {
        cout << "No members available.\n";
        return;
    }
    for (const auto& m : members) {
        m.display();
        cout << "--------------------\n";
    }
}

void searchMember() const {
    int id = readInt("Enter Member ID to search: ");
    int index = memberIndex(id);

    if (index == -1)
        cout << "Member not found.\n";
    else
        members[index].display();
}

void addEvent() {
    int id = readInt("Enter Event ID: ");
    if (eventIndex(id) != -1) {
        cout << "Event ID already exists.\n";
        return;
    }

    string name = readText("Enter Event Name: ");
    string club = readText("Enter Organizing Club: ");
```

```
    if (!clubExists(club)) {
        cout << "Club not found. Add the club first.\n";
        return;
    }

    string date = readText("Enter Date (DD/MM/YYYY): ");
    string venue = readText("Enter Venue: ");

    events.emplace_back(id, name, club, date, venue);
    saveData();
    cout << "Event added successfully!\n";
}

void showEvents() const {
    if (events.empty()) {
        cout << "No events available.\n";
        return;
    }
    for (const auto& e : events) {
        e.display();
        cout << "--------------------\n";
    }
}

void registerEvent() {
    int mid = readInt("Enter Member ID: ");
    int eid = readInt("Enter Event ID: ");

    int mi = memberIndex(mid);
    int ei = eventIndex(eid);
```

```
    if (mi == -1 || ei == -1) {
        cout << "Invalid member ID or event ID.\n";
        return;
    }

    if (members[mi].getClub() != events[ei].getClub()) {
        cout << "Only members of the organizing club can register.\n";
        return;
    }

    if (registered(mid, eid)) {
        cout << "Already registered for this event.\n";
        return;
    }

    registrations.emplace_back(mid, eid, false);
    saveData();
    cout << "Event registration successful!\n";
}

void markAttendance() {
    int eid = readInt("Enter Event ID: ");
    if (eventIndex(eid) == -1) {
        cout << "Event not found.\n";
        return;
    }

    int mid = readInt("Enter Member ID: ");

    for (auto& r : registrations) {
        if (r.memberId == mid && r.eventId == eid) {
```

```
            int choice = readInt("1. Present  2. Absent: ");
            if (choice != 1 && choice != 2) {
                cout << "Invalid choice.\n";
                return;
            }
            r.present = (choice == 1);
            saveData();
            cout << "Attendance updated.\n";
            return;
        }
    }
    cout << "Member is not registered for this event.\n";
}

void showReport() const {
    cout << "\n===== COLLEGE CLUB REPORT =====\n";
    cout << "Total Clubs: " << clubs.size() << '\n';
    cout << "Total Members: " << members.size() << '\n';
    cout << "Total Events: " << events.size() << '\n';
    cout << "Total Registrations: " << registrations.size() << '\n';

    int present = 0;
    for (const auto& r : registrations)
        if (r.present) present++;

    cout << "Members Marked Present: " << present << '\n';

    if (!registrations.empty()) {
        double percentage = 100.0 * present / registrations.size();
        cout << "Overall Attendance: " << percentage << "%\n";
    }
```

```
    }

public:
    CollegeClubSystem() {
        loadData();
    }

    void run() {
        int choice;
        do {
            cout << "\n====================================\n";
            cout << "   COLLEGE CLUB MANAGEMENT SYSTEM\n";
            cout << "====================================\n";
            cout << "1. Add Club\n";
            cout << "2. Display Clubs\n";
            cout << "3. Add Member\n";
            cout << "4. Display Members\n";
            cout << "5. Search Member\n";
            cout << "6. Add Event\n";
            cout << "7. Display Events\n";
            cout << "8. Register for Event\n";
            cout << "9. Mark Attendance\n";
            cout << "10. View Report\n";
            cout << "0. Exit\n";

            choice = readInt("Enter your choice: ");

            switch (choice) {
                case 1: addClub(); break;
                case 2: showClubs(); break;
                case 3: addMember(); break;
```

```
            case 4: showMembers(); break;
            case 5: searchMember(); break;
            case 6: addEvent(); break;
            case 7: showEvents(); break;
            case 8: registerEvent(); break;
            case 9: markAttendance(); break;
            case 10: showReport(); break;
            case 0:
                saveData();
                cout << "Data saved. Goodbye!\n";
                break;
            default:
                cout << "Invalid choice. Try again.\n";
        }
    } while (choice != 0);
  }
};

int main() {
  CollegeClubSystem system;
  system.run();
  return 0;
}
```